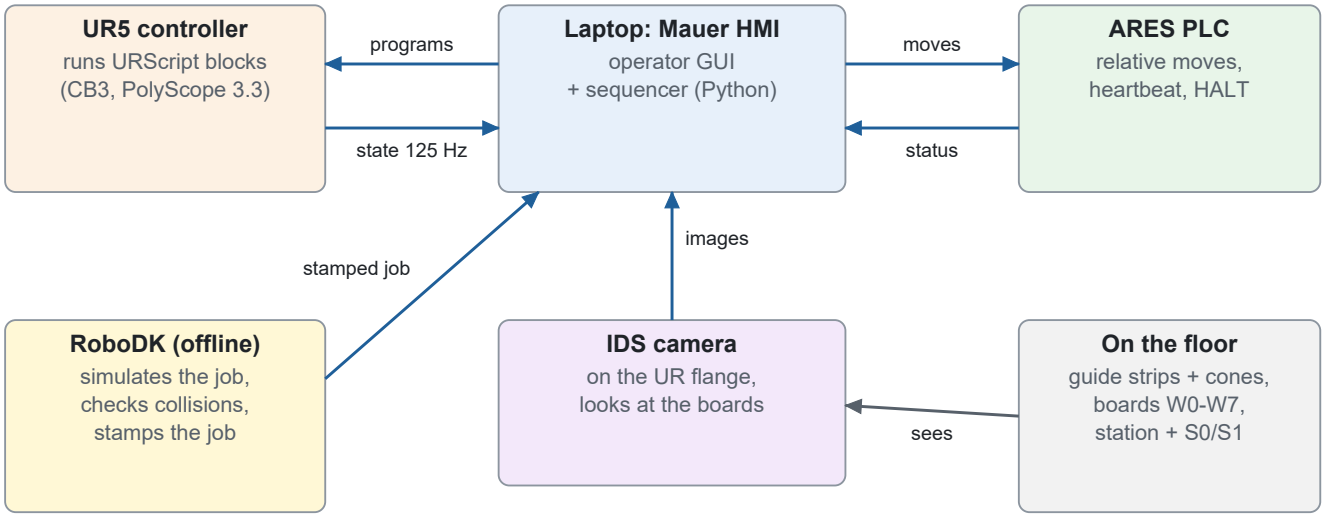
ARES Mauer: how the system works

Concepts, not code. State of main 766097a, 15 October 2026 (C wall, 1 mm head joint, as-laid guide gaps, release 20 mm, ARES tour).

1 The idea

A UR5 arm stands on the mobile platform ARES. ARES carries a magazine with up to 18 stones and drives to a few fixed **stops** along a C-shaped wall of interlocking stones (pins up, 4 courses, 67 stones). At each stop the arm lays the stones it can reach. When the magazine is empty, ARES drives to a **pick-up station** on the floor, the arm refills the magazine, and ARES drives back.

The core problem is accuracy. ARES drives to within a few centimetres, but the stones need about a millimetre. The system therefore never trusts where ARES *should* be. At every stop the camera on the arm looks at printed reference boards on the floor and measures where the wall really is, relative to the robot. Every stone is then placed relative to that measurement.



One Ethernet switch: laptop, UR 192.168.56.101, ARES 192.168.1.10, camera 192.168.50.10.

Figure 1: The parts and how they talk. The laptop is the only brain; the UR and the ARES PLC execute single commands.

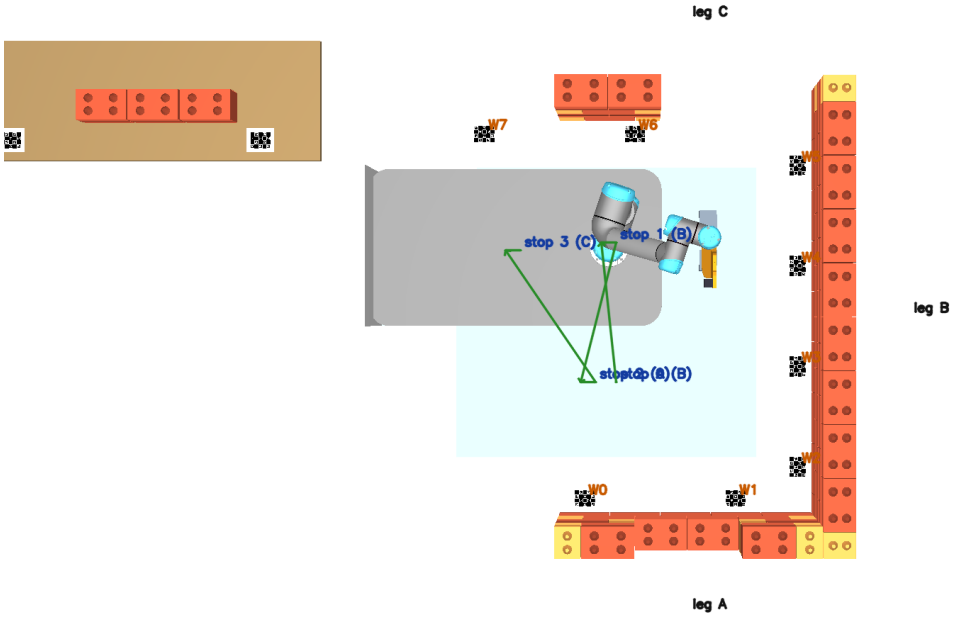


Figure 2: The finished C in the RoboDK simulation: legs A, B, C, the boards W0-W7 in front of the legs, ARES (grey) with the arm, its four stops (blue), the routes between them (green) and the pick-up station (top left).

2 Plan offline, execute online

Everything that can be decided in advance is decided once, on the laptop, and written into a **job file**. The run itself only executes that job and corrects for what it measures.

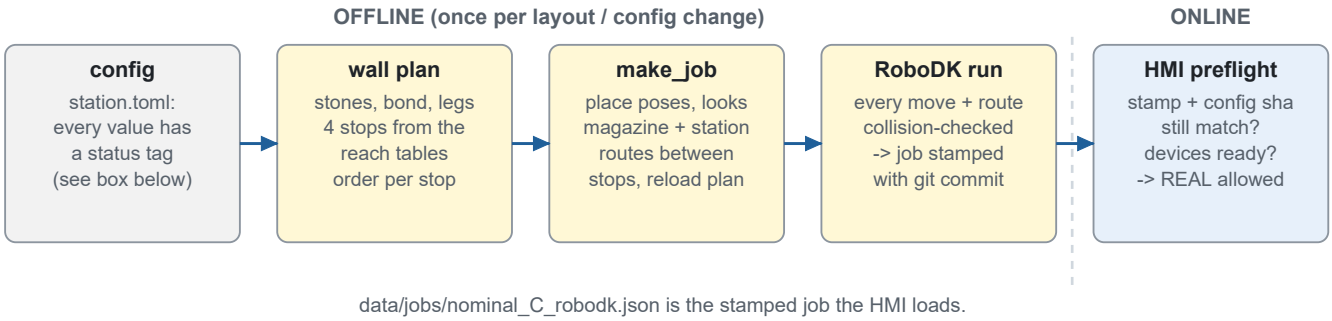


Figure 3: From the configuration to a run.

What the job contains

- **Stops**: 4 nominal ARES poses (stop 0 and 1 at leg B, stop 2 at A, stop 3 at C) with the stones of each stop in laying order (26, 12, 19, 10).
- **Stones**: for each one the place pose in the wall frame, the magazine slot, a joint-angle hint for the robot, and whether it comes down from the side (next to a placed neighbour).
- **Looks**: where the arm holds the camera to see the boards of each stop.
- **Magazine and station**: every slot, its stone type (full / half) and the order of taking and filling; the 4 station trips fall where the magazine runs empty.
- **Routes**: waypoint lists for every ARES drive (stop to stop, stop to station and back), each leg either a pure translation or a pure rotation, checked against the floor model.

Why the stamp matters

RoboDK runs the complete job once (every pick, place, transfer, look, park and every ARES route) and checks it for collisions with CAD models. Only a clean run writes the **stamp**: the git commit of the code and the config it verified. Before a REAL run the HMI compares the stamp with the code and config on disk. If planning-relevant code or config changed, REAL is refused until RoboDK has run again. Tables that are not planning input ([hmi], [magtest], [laid]) are ignored, so an as-laid floor correction never needs a new RoboDK run.

Status tags. Every number in config/station.toml says where it comes from: CONFIRMED (measured or decided), ASSUMPTION (a reasonable guess) or PLACEHOLDER (unknown). Values that directly move the arm without a camera check (UR mount height and angle, magazine holder height, board scale, settle time, payload CoG) must not be PLACEHOLDER for a REAL run.

3 Where is the wall? The camera loop

The most important idea of the system is one equation:

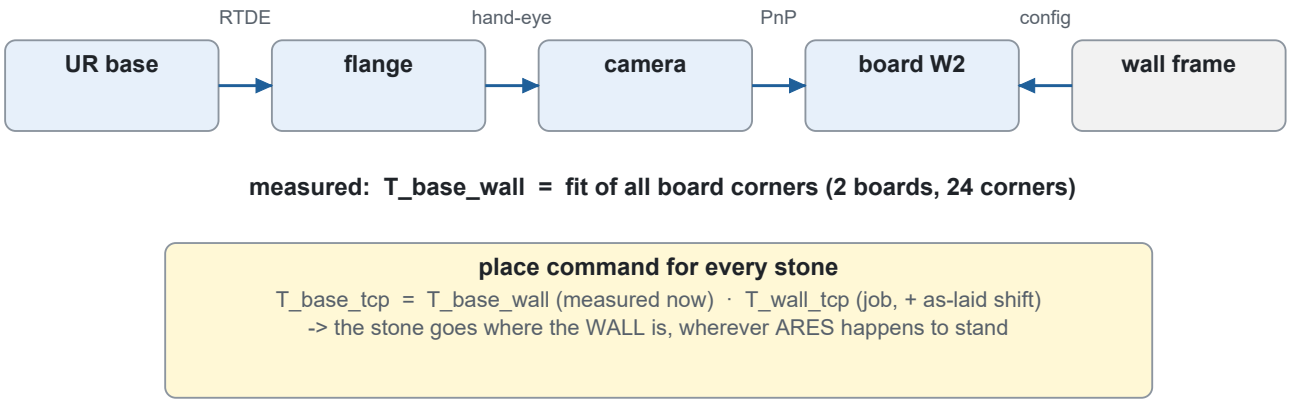


Figure 4: The measurement chain (top) and how a place command is formed (bottom).

How one measurement works

- The arm moves to a **look pose** about 320 mm above a board and waits 1 s for ARES to stop swaying.
- The camera takes an image. At the same time the UR reports its joint angles 125 times a second; the samples during the exposure give the exact flange pose at the moment of the image.
- OpenCV finds the ChArUco board (5 x 4 squares of 16 mm, 12 inner corners). A board counts only with at least 8 corners and a reprojection error below 1 pixel. PnP gives the board pose relative to the camera.
- The **hand-eye calibration** (2026-10-09, 17 poses, about 0.7 mm / 0.4° hold-out error) links camera and flange. Chained together: where the board lies in the UR base frame.
- Two boards per stop, 600 to 1200 mm apart, are fitted together. The fit gives the full 6-DoF wall frame, including the slight tilt of ARES on its springs (about 0.7° measured). The long distance between the boards makes the heading accurate.

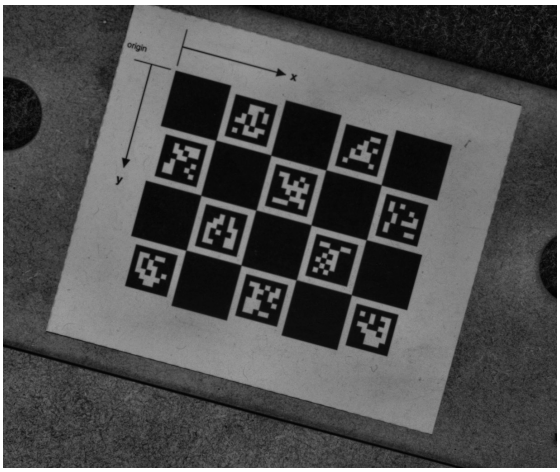


Figure 5: What the camera saw at stop 0 on 15 October (board W2, contrast enhanced).

Two built-in plausibility checks

- **Fit residual <= 1.0 mm**: the corners of both boards must agree with their known relative placement. If not, a board moved, the floor is laid differently than planned, or the calibration is off. The run pauses with the arm parked. (This is what caught the 1 mm gaps between the guide pieces.)
- **Jump check**: the measured ARES pose may differ from the predicted one by at most 50 mm / 3° (100 mm / 5° after a route or a station trip). A larger jump means a wrong board or a big slip, and the run stops with an error.

As-laid correction

The guide strips are cut into pieces joined by dovetails. On the floor they lie with a 1 mm gap at every joint, so every piece after a joint sits 1 mm further along its leg than drawn. The setting [laid] piece_gap_mm = 1.0 moves each board and each stone by the gaps before its piece (up to 3.2 mm). This happens at run time, so the RoboDK-verified job stays unchanged.

4 Local corrections: when does ARES move again?

Because the stones are placed relative to the measured wall, ARES does not have to stand exactly on its stop. It only has to stand close enough that every stone of the stop is still within reach, with the same margin the planning used. Hence the rules:

Situation	Rule	Why
Measured within 20 mm and 0.5° of the stop	no ARES move	reach tables were checked with a ±20 mm margin; 0.5° is about 3.5x the rotation scatter of ARES
Further off	park the arm, one correction move, measure again (at most 2 corrections)	get back into the verified reach
Arriving after a route	the route stops 40 mm short of the stop; the camera measures; the correction closes the rest	ARES never drives blind towards the board plates (110 mm away)
At the station dock	correct if more than 30 mm off	the station stacks tolerate 30 mm and 1.5°
Any ARES move	only with the arm parked; moves below 2 mm / 0.2° are not sent	the routes were checked with the parked arm; the camera absorbs tiny errors

ARES itself moves by relative commands to its PLC: *translate dx, dy* or *rotate θ*, never both at once, at 150 mm/s and 10°/s. The PLC closes its loop on wheel odometry; wheel slip is invisible to it, which is exactly what the camera loop catches. Because the UR sits at the vehicle rear, the software frame is turned 180° against the vehicle; every confirmation therefore also names the physical direction (for example "= vehicle 300 mm BACKWARD").

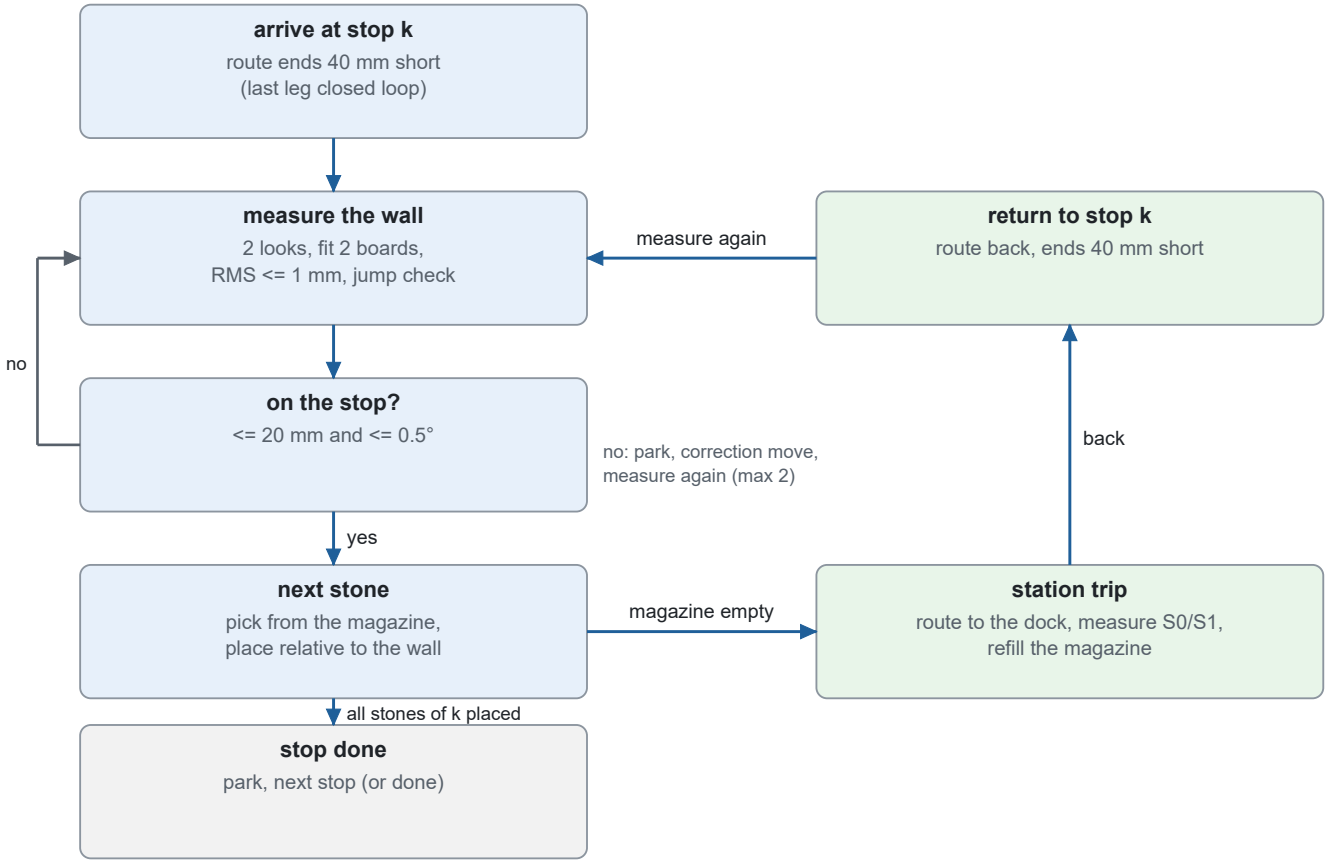


Figure 6: One stop. The station trip happens whenever the magazine runs empty, here 4 times in the whole wall.

5 Robot motions

The laptop never streams motion. For each action (look, park, pick, place) it generates one small **URScript program**, sends it to the controller and watches the controller state until the program reports "done". Each program first sets the tool and the payload, then checks that every target pose is reachable (if not, it stops before moving), then moves.

Pick and place

- Joint move to 150 mm above the target (the joints are planned and collision-checked first).
- Straight down fast to 60 mm above, then slowly (20 mm/s) through the contact zone.
- **Pick**: close the jaws (0.5 s pulse), switch the payload to tool + stone, lift.
- **Place**: the jaws open **20 mm above** the seated position and the stone drops onto the cones or pins below; pressing it down had caused a protective stop.
- A stone next to an already placed neighbour comes down 20 mm to the side, 30 mm above the pins, and slides over before the last drop (57 of 67 stones).
- Speed: the HMI's REAL speed (default 25 %) scales all speeds; at 100 % the slow zone takes 3 s.

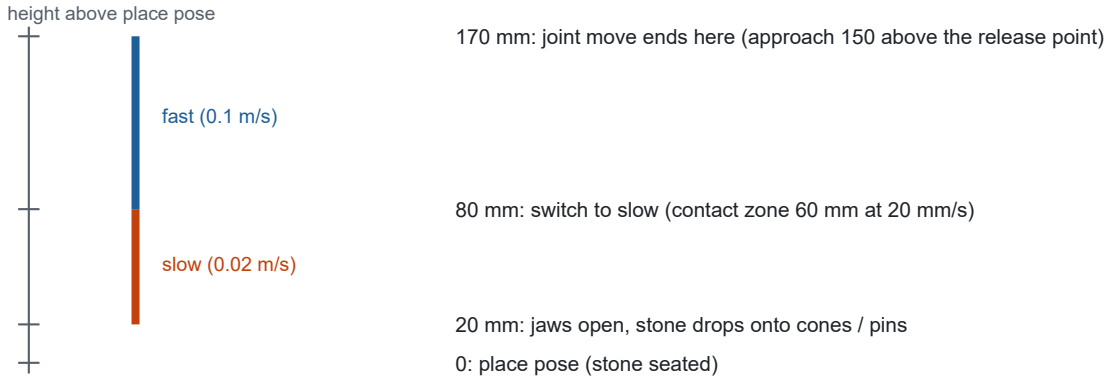


Figure 7: The vertical part of a place (wall or magazine).

6 Collisions: two independent checks

	RoboDK verification (offline)	Motion guard (online)
When	once, before any real run	before every robot program
Model	exact CAD meshes	simple capsules and boxes, 10 mm clearance (15 mm tool to arm)
World	nominal ARES poses	the measured wall, the stones actually standing, the filled magazine slots, the station when docked
Paths	plans its own paths to prove the job is feasible	plans the path that is really sent: direct, via park, via a pose above the target, ... and if none is clear a random search (RRT-Connect)
If blocked	no stamp, REAL refused	"refused by the motion guard": nothing is sent, the run stops

Neither checks the last 60 mm at a stone: that is deliberate contact. ARES routes are checked against a floor model: the ARES footprint including the controller box (1370 x 600 mm, rotation circle about 864 mm), the legs, the board plates and the station, with 50 mm clearance.

7 Safety and operator control

- **E-stops** on ARES and on the UR pendant are the only safety function. Everything else is operation.
- **HALT** (button, Space or Esc, while the HMI window is active): ARES stops with a ramp and stays in MANUAL, the UR program is stopped, nothing new is sent until Start or Resume.
- **Step mode**: every robot program and every ARES move needs **Go**; **Decline** stops the run in a resumable state.
- **Heartbeat**: the HMI pings the PLC every 100 ms. If it stops (HMI crashed), the PLC aborts a move after 500 ms and leaves MANUAL after 2 s.
- **Interlocks**: ARES moves only with the arm parked and idle; no robot program while another one runs; Resume is refused while a stone may be in the jaws or the ARES pose is unverified.
- **Preflight**: before REAL the HMI checks job stamp, config, calibrations, UR state, ARES state and camera; there is no override.

8 The HMI

Started with **.\hmi.cmd --ares --job data/jobs/nominal_C_robodk.json**. Without --ares only SIM is possible. Always visible on every tab:

- the red **HALT** button (Space / Esc) and, if another window has the keyboard, a red banner saying keyboard HALT is inactive;
- the **status strip**: run state, SIM/REAL, stop, stone, current action, UR, ARES and twin state;

- the yellow **confirm bar** when something needs an answer: a step (Go / Decline), the start checklist (Checked - start / Not ready) or an empty station (Refilled / Abort run).

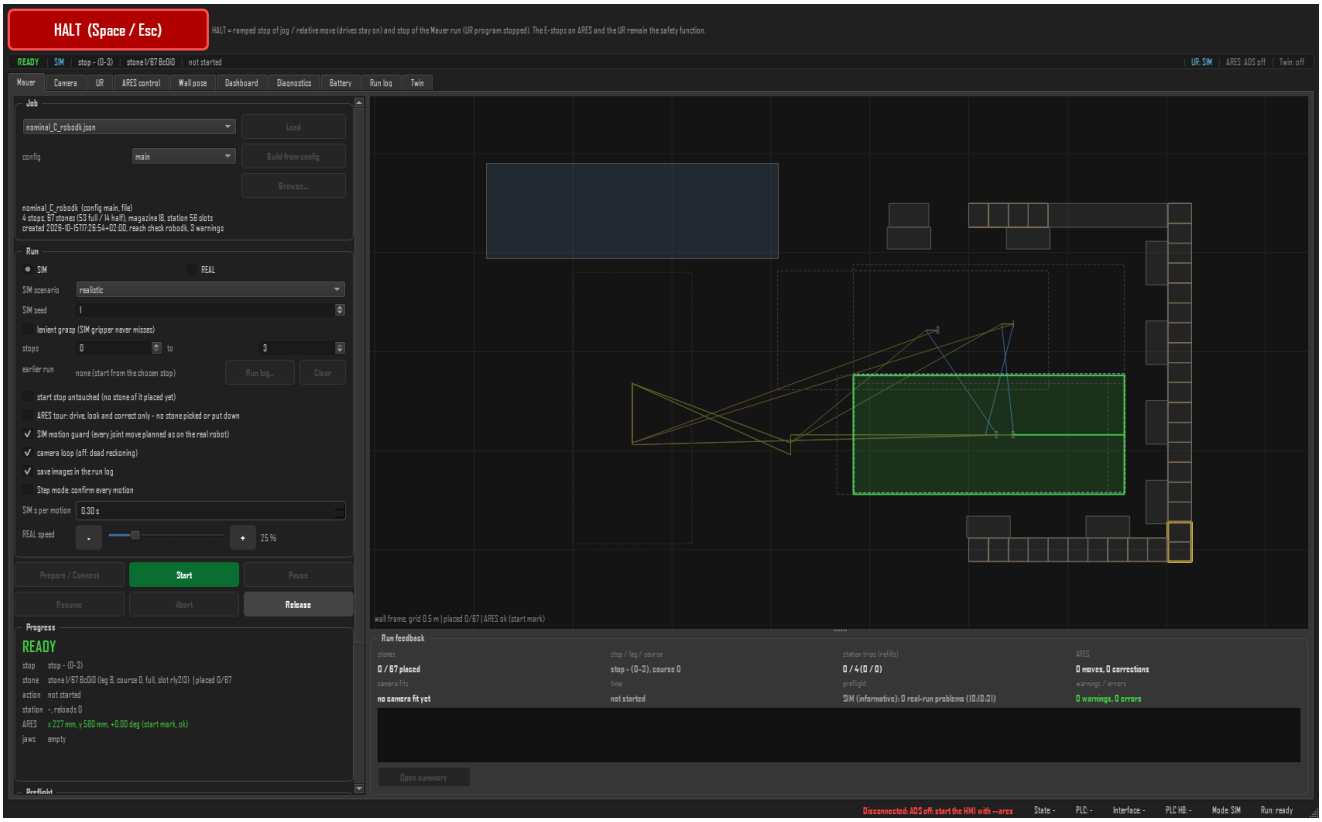


Figure 8: Mauer tab with the job loaded and the SIM rig prepared (READY). Left: job, run options, buttons, progress. Right: the plan view (wall frame) with ARES at stop 0 highlighted, routes, station; below it the run feedback tiles.

Mauer tab, top to bottom

- **Job**: pick and Load a job, or Build from config.
- **Run**: SIM or REAL; stops from-to; *earlier run* (a run log whose stones already stand) or *start stop untouched*; *ARES tour* (drive and measure only, no stones); SIM options (scenario, seed, motion guard); camera loop; save images; **step mode**; REAL speed (5-100 %, changeable during the run).
- **Buttons**: Prepare / Connect (opens UR, ADS, camera, runs the preflight), Start, Pause (takes effect when the jaws are empty), Resume, Abort, Release (closes everything).
- **Progress**: state, stop, stone, current action, ARES pose (green measured, orange odometry only, red unknown), jaws.
- **Recovery** (only when stopped): Confirm pose, Set pose, Apply odometry, Jaws empty, Re-check.
- **Preflight** list: red lines block REAL; "REAL preflight ok" in green when everything is fine.

Run states

loaded → (Prepare) → ready → (Start) → running → paused / aborted / error (resumable) or done → (Release) → loaded. After an error or a HALT, step mode switches on for the resume.

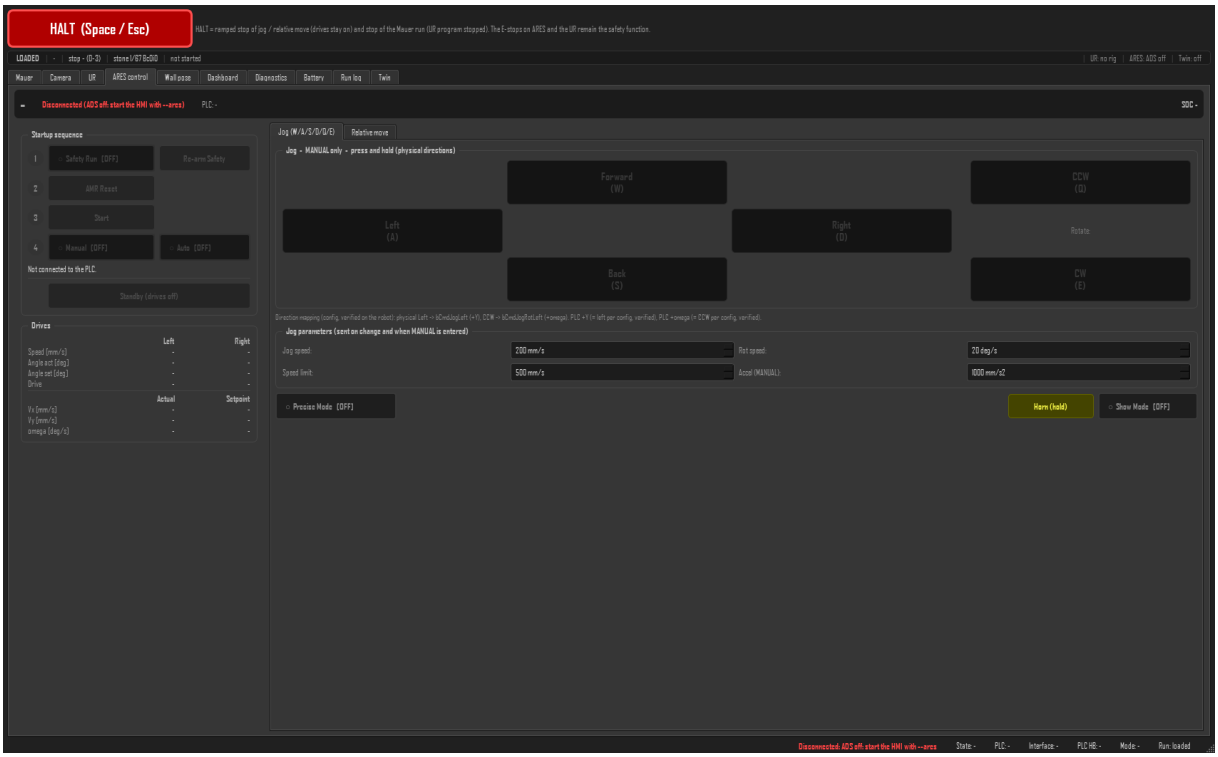


Figure 9: ARES control tab (here without a PLC connection). Left: the start sequence Safety Run → AMR Reset → Start → Manual and the drive values; right: jog (press and hold) and the relative-move panel.

The other tabs

Tab	What it shows
Camera	the last image with the detected corners per board (green accepted, red rejected) and the last frame fit: RMS, error to the nominal stop, jump against the prediction; Grab / Live in REAL when no run is active
UR	RTDE age, robot and safety mode, joints, TCP, gripper outputs and the payload last set
ARES control	start sequence to MANUAL, jog, relative move (GO needs a second click within 3 s), odometry map
Wall pose	plan view plus the ARES estimate, target, PLC odometry, moves, and a table of all camera fits
Dashboard, Diagnostics, Battery	ARES overview, raw PLC variables, battery charge
Run log	every event of the run (motion, vision, stones, warnings), filterable
Twin	optional RoboDK twin that follows the run (it never controls anything)

Typical situations

You see	Do
"a stone may be in the jaws" on Resume	hold the stone, open the gripper on the pendant (DO0 pulse), take it out, move the arm clear, **Jaws empty**, Resume
"ARES pose is unverified" (orange)	check ARES on the floor, **Confirm pose** (or Set pose)
Fit RMS > 1 mm, run paused	a board moved or the floor differs from the plan; fix, then Resume (measures again)
Confirmation direction looks wrong	**Decline**: away from wall/station = FORWARD, towards = BACKWARD, along a leg = LEFT/RIGHT
UR program paused after an E-stop	press Stop on the pendant, then Re-check and Start
HMI closed or crashed	the run state is lost; put ARES on the start mark of stop k, start again with the run log of the interrupted run (or 'stop untouched')

9 Testing without risk

- **SIM**: the same sequencer runs against a simulated world with realistic errors (wheel slip, drive scatter, mount and station offsets) and rendered camera images. With the motion guard on, the whole C runs 67/67 with 0 collisions (3 seeds, max placement error about 2.6 mm).
- **ARES tour**: on the real robot, the full route of the wall run (34 ARES moves, 4 station trips) with every look and correction, but no stone is touched. **tools/tour_report.py** then lists, for every measurement, how far ARES ended from where it was predicted (the real drive error).
- **Magazine dry run**: the arm cycles stones in the magazine while ARES stands still.

In one sentence: plan everything once and verify it in RoboDK; at run time trust only the camera's measurement of the wall, move ARES only when it is outside the verified margin, check every arm path again just before sending it, and keep a human with Go, HALT and the E-stop in the loop.